# ППОИС
## Часть 1

Автоматное программирование

# 1. Регулярные выражения

- **Регуля́рные выраже́ния** (англ. regular expressions, regex) — формальный язык поиска и осуществления манипуляций с подстроками в тексте, основанный на использовании метасимволов (символов-джокеров, англ. wildcard characters).
- По сути это строка-образец (англ. pattern, по-русски её часто называют «шаблоном», «маской»), состоящая из символов и метасимволов и задающая правило поиска.

# 2. Части строк

- 1. *Префиксом (prefix) строки s является любая строка, полученная удалением нуля или нескольких последних символов строки s.*

- *2. Суффиксом (suffix) строки s является любая строка, полученная удалением нуля или нескольких первых символов строки s.*

# 3. Части строк (2)

- 3. *Подстрока (substring) строки s получается путем удаления произвольного префикса и произвольного суффикса из строки s.*
- 4. *Правильными, или истинными (proper), префиксами, суффиксами и подстроками s являются соответственно префиксы, суффиксы и подстроки s, которые не являются пустыми строками и не совпадают с самой строкой s.*
- 5. *Подпоследовательностью (subsequence) строки s является любая строка, образованная удалением нуля или нескольких не обязательно смежных позиций из строки s.*

# 4. Операции над языками

| Операция | Определение и обозначение |
|---|---|
| *Объединение* (union) $L$ и $M$ | $L \cup M = \{s \mid s \in L$ или $s \in M\}$ |
| *Конкатенация* (concatenation) $L$ и $M$ | $LM = \{st \mid s \in L$ и $t \in M\}$ |
| *Замыкание Клини* (Kleene closure) языка $L$ | $L^* = \bigcup_{i=0}^{\infty} L^i$ |
| *Позитивное замыкание* (positive closure) языка $L$ | $L^+ = \bigcup_{i=1}^{\infty} L^i$ |

# 5. Правила задания языков

1. $(r) \mid (s)$ – регулярное выражение, описывающее язык $L(r) \cup L(s)$.
2. $(r)(s)$ – регулярное выражение, описывающее язык $L(r)L(s)$.
3. $(r)^*$ – регулярное выражение, описывающее язык $(L(r))^*$.
4. $(r)$ – регулярное выражение, описывающее язык $L(r)$. Это правило говорит о том, что можно заключить выражение в скобки без изменения описываемого им языка.

# 6. Законы для регулярных выражений

| Закон | Описание |
|---|---|
| $r \mid s = s \mid r$ | Оператор \| коммутативен |
| $r \mid (s \mid t) = (r \mid s) \mid t$ | Оператор \| ассоциативен |
| $r\,(st) = (rs)\,t$ | Конкатенация ассоциативна |
| $r\,(s \mid t) = rs \mid rt$; $(s \mid t)\,r = sr \mid tr$ | Конкатенация дистрибутивна над \| |
| $\epsilon r = r\epsilon = r$ | $\epsilon$ является единичным элементом по отношению к конкатенации |
| $r^* = (r \mid \epsilon)^*$ | $\epsilon$ гарантированно входит в замыкание |
| $r^{**} = r^*$ | Оператор * идемпотентен |

# 7. Расширения

- 1. Один или несколько a+
- 2. Один или ни одного a?
- 3. a{n} точно n раз
- 4. a{n,} не менее n раз
- 5. a{n,m} не менее n, но не более m раз
- 5. Класс символов 1..9, a..z

# 8. Конечный автомат

- **Конечный автомат (finite-state machine, FSM)** — это математическая абстракция, модель, которая может находиться только в одном из конечного числа состояний в каждый конкретный момент времени.
- Автомат умеет переходить из одного состояния в другое в ответ на данные, которые подаются на вход; изменение состояния называется переходом.

# 9. Конечный автомат (2)

$M = (Q, \Sigma, \phi, q_0, F)$

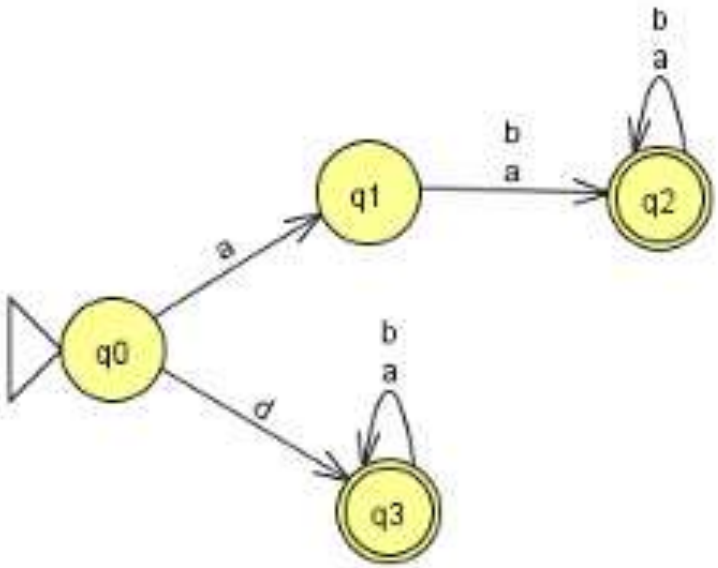
- Q - конечное множество *состояний*;
- Σ - конечное множество допустимых *входных символов*;
- ϕ - отображение множества $Q \times \Sigma$ в множество P(Q), определяющее поведение управляющего устройства; функцию ϕ иногда называют *функцией переходов*;
- $q_0 \in Q$ – *начальное состояние* управляющего устройства;
- $F \subseteq Q$ – множество *заключительных* состояний.

# 10. Способы задания

- Таблица состояний

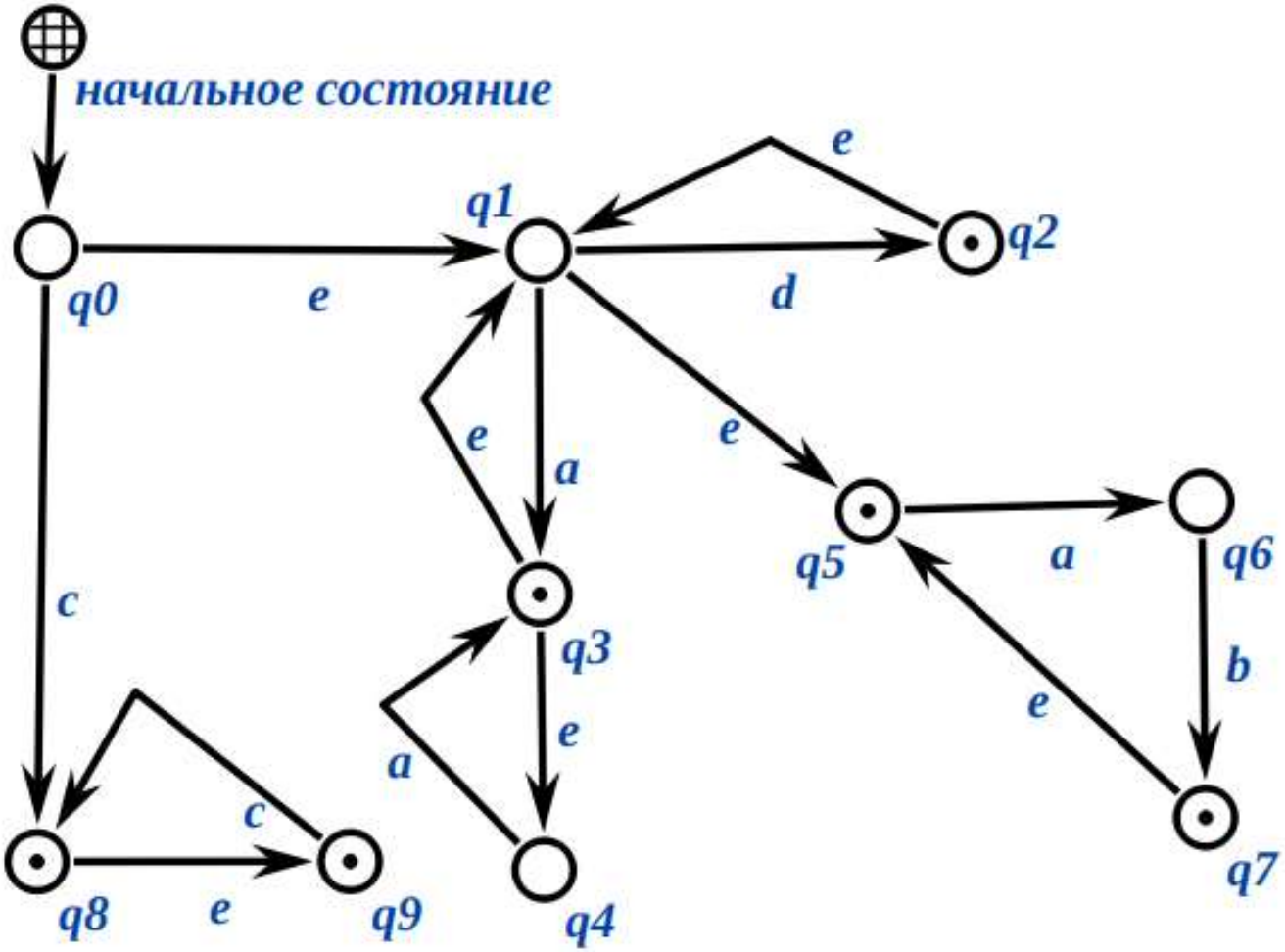
| | a | b | d |
|---|---|---|---|
| q0 | q1 | | q3 |
| q1 | q2 | q2 | |
| q2 | q2 | q2 | |
| q3 | q3 | q3 | |

- Диаграмма переходов



# 11. Пример

- Пример: **(d|a*)*(ab)*|c***

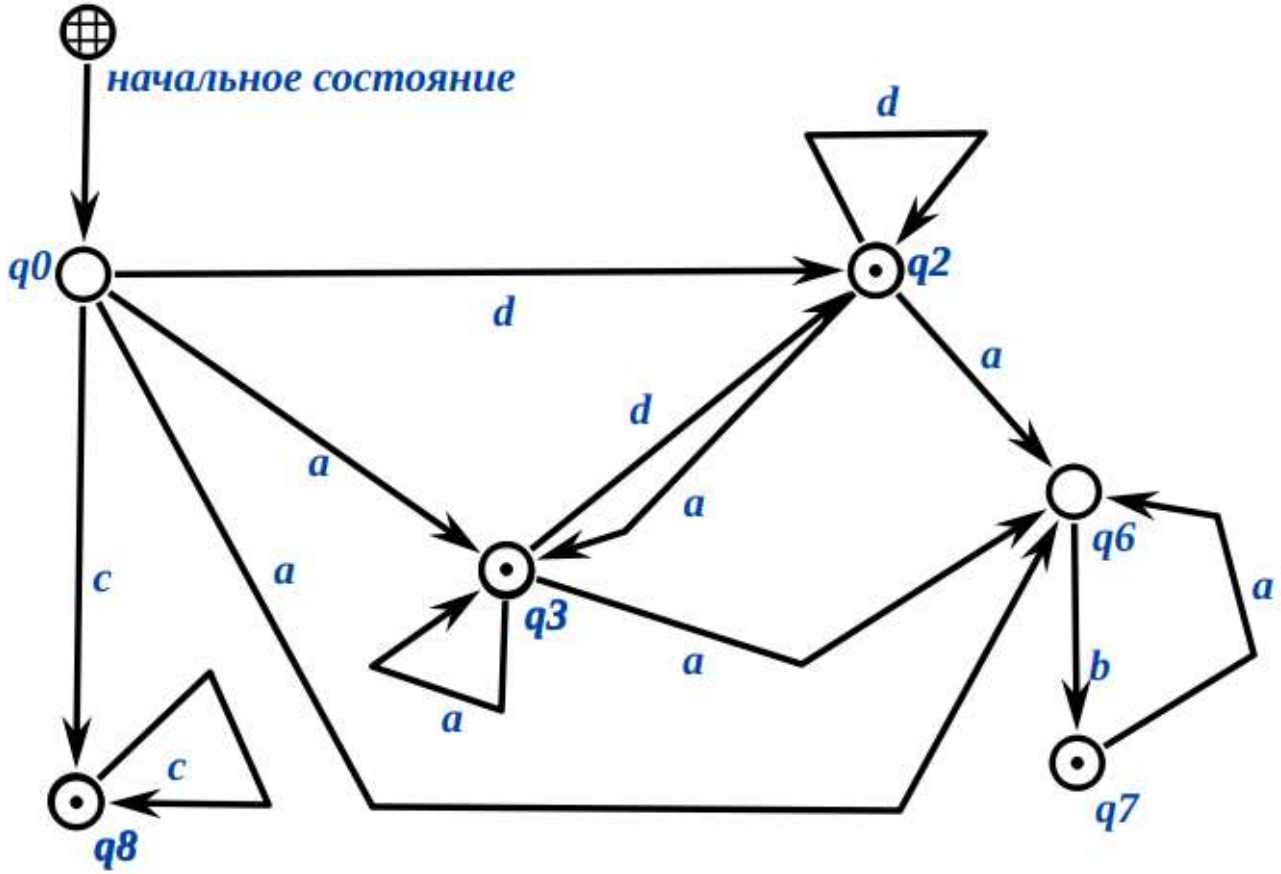


# 11. Детерминированный конечный автомат

- Нет переходов для ε;
- Для каждого состояния s и входного символа a имеется ровно одна дуга, выходящая из s и помеченная a;

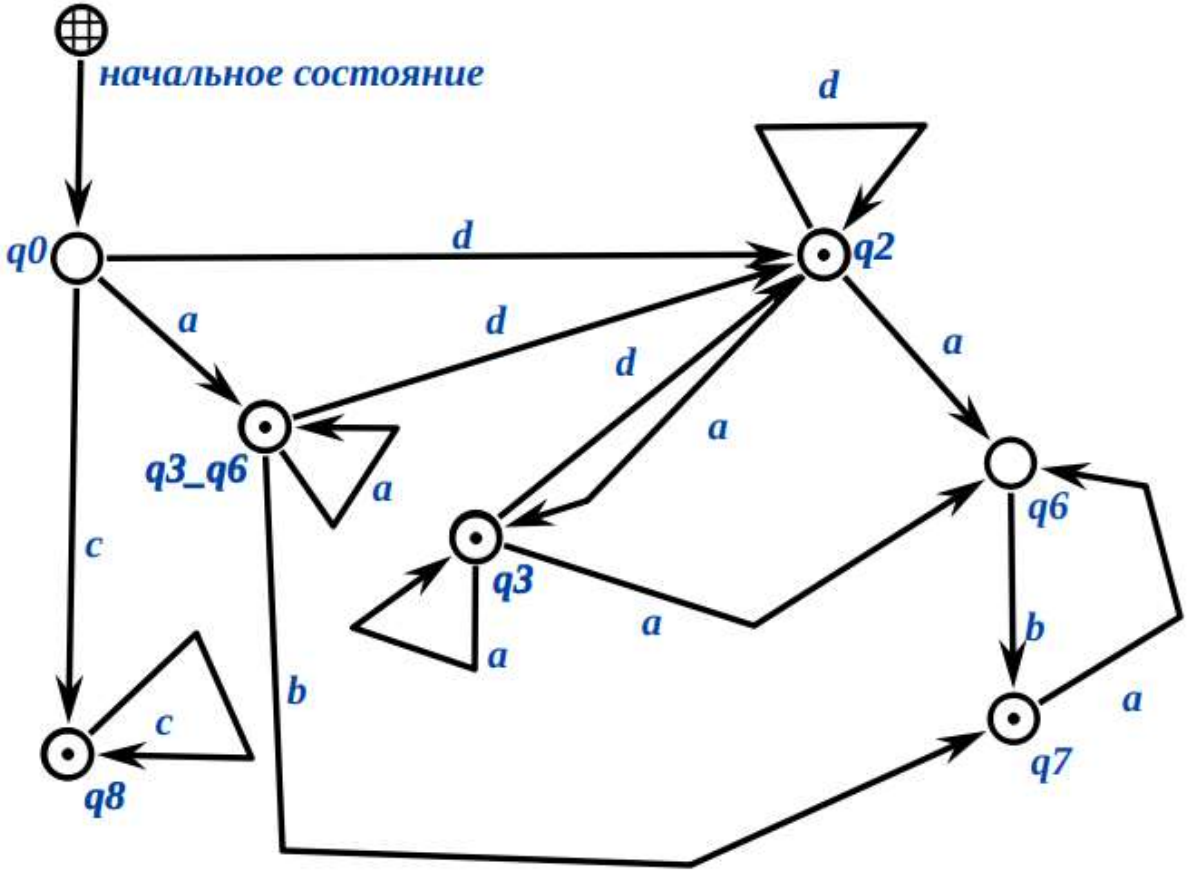
# 12. Пример без е-переходов

- Пример: **(d|a*)*(ab)*|c***



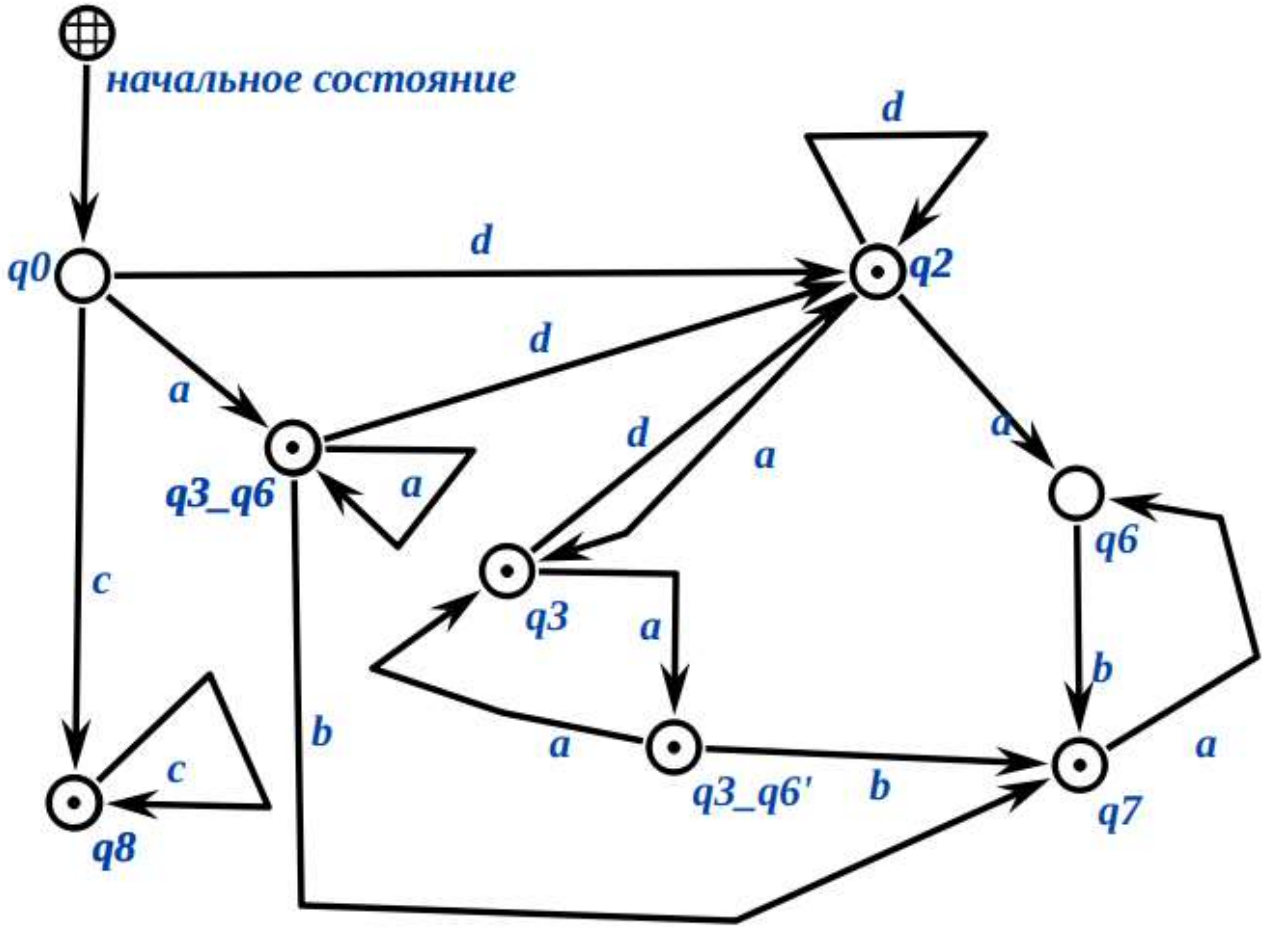
# 13. Детерминизация(1)

- Пример: **(d|a*)*(ab)*|c***



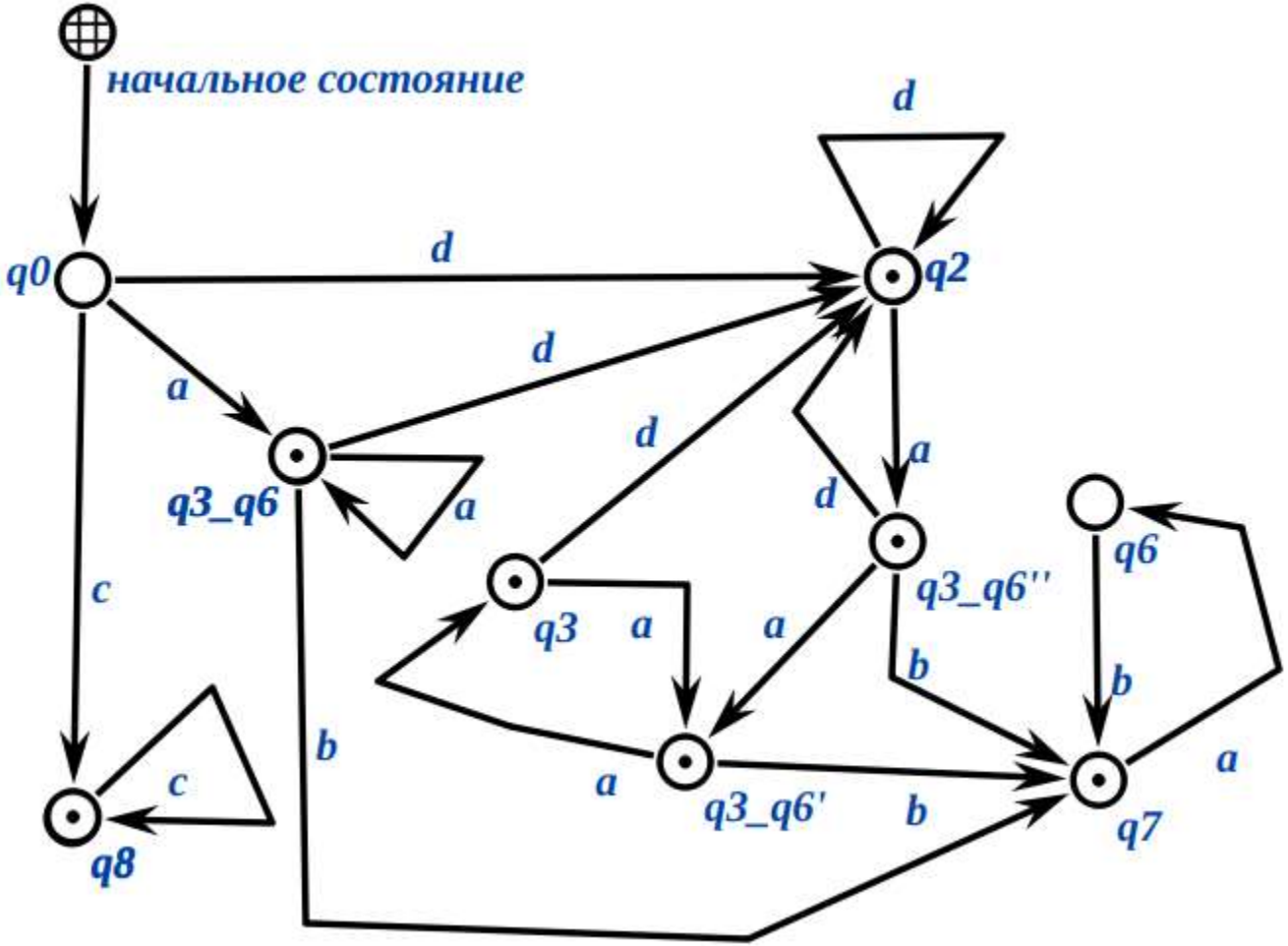
# 14. Детерминизация(2)

- Пример: **(d|a*)*(ab)*|c***



# 15. Детерминизация(3)

- Пример: **(d|a*)*(ab)*|c***



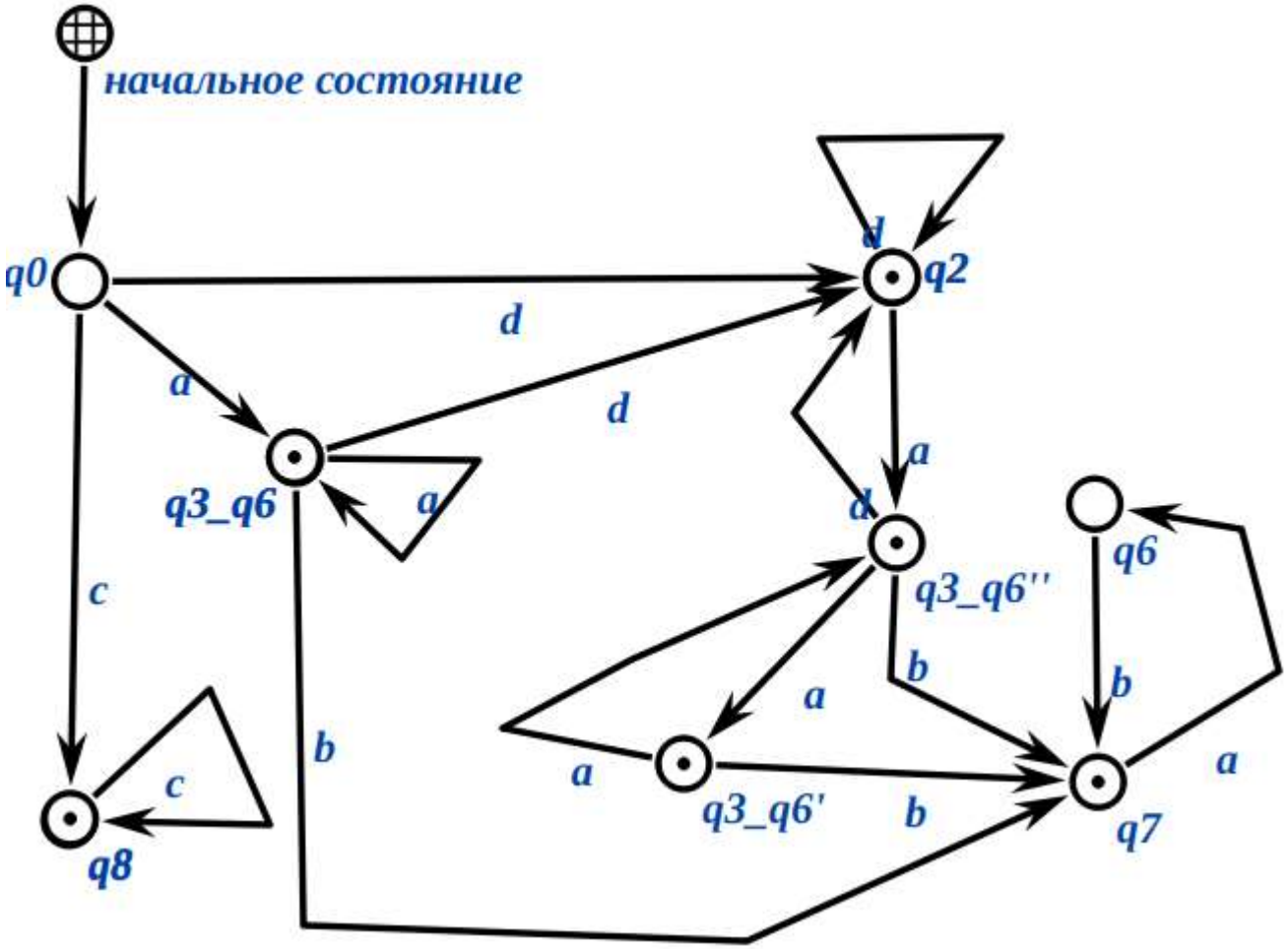
# 16. Таблица состояний

- Пример: **(d|a*)*(ab)*|c***

| | a | b | c | d |
|---|---|---|---|---|
| q0 | q3_q6 | | q8 | q2 |
| q2 | q3_q6'' | | | q2 |
| q3 | q3_q6' | | | q2 |
| q6 | | q7 | | |
| q7 | q6 | | | |
| q8 | | | q8 | |
| q3_q6 | q3_q6 | q7 | | q2 |
| q3_q6' | q3 | q7 | | |
| q3_q6'' | q3_q6' | q7 | | q2 |

# 17. Минимизация

- Пример: **(d|a*)*(ab)*|c***



# 18. Тест

$$d|a^2(ab)^*c$$

# Диаграмма состояний

State diagram

# Цель

- **Диаграмма состояний** (state diagram) - диаграмма, которая представляет *конечный автомат*.
- **Конечный автомат** (state machine) - модель для спецификации поведения объекта в форме последовательности его состояний, которые описывают реакцию объекта на внешние события, выполнение объектом действий, а также изменение его отдельных свойств.

# Конечный автомат

$M = (Q, \Sigma, \phi, q_0, F)$

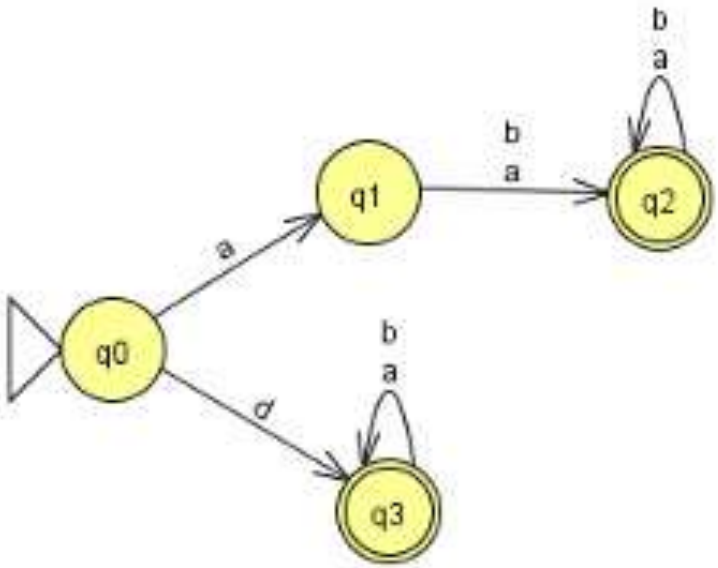
- $Q$ - конечное множество *состояний*;
- $\Sigma$ - конечное множество допустимых *входных символов*;
- $\phi$ - отображение множества $Q \times \Sigma$ в множество $P(Q)$, определяющее поведение управляющего устройства; функцию $\phi$ иногда называют *функцией переходов*;
- $q_0 \in Q$ – *начальное состояние* управляющего устройства;
- $F \subseteq Q$ – множество *заключительных* состояний.

# Способы задания

- Таблица состояний

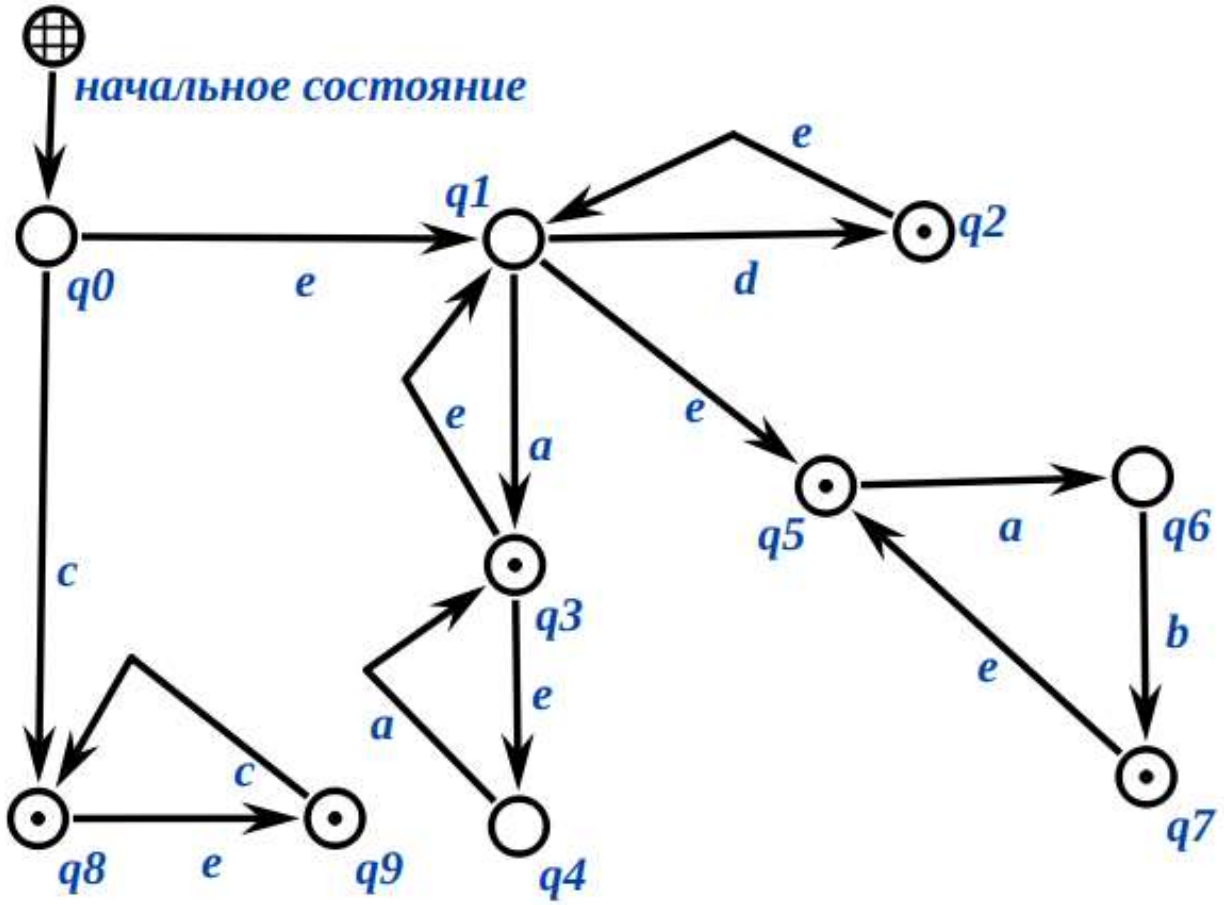
| | a | b | d |
|---|---|---|---|
| q0 | q1 | | q3 |
| q1 | q2 | q2 | |
| q2 | q2 | q2 | |
| q3 | q3 | q3 | |

- Диаграмма переходов



# Пример

- daaaaababababcccccc
- Пример: **(d|a*)*(ab)*|c***

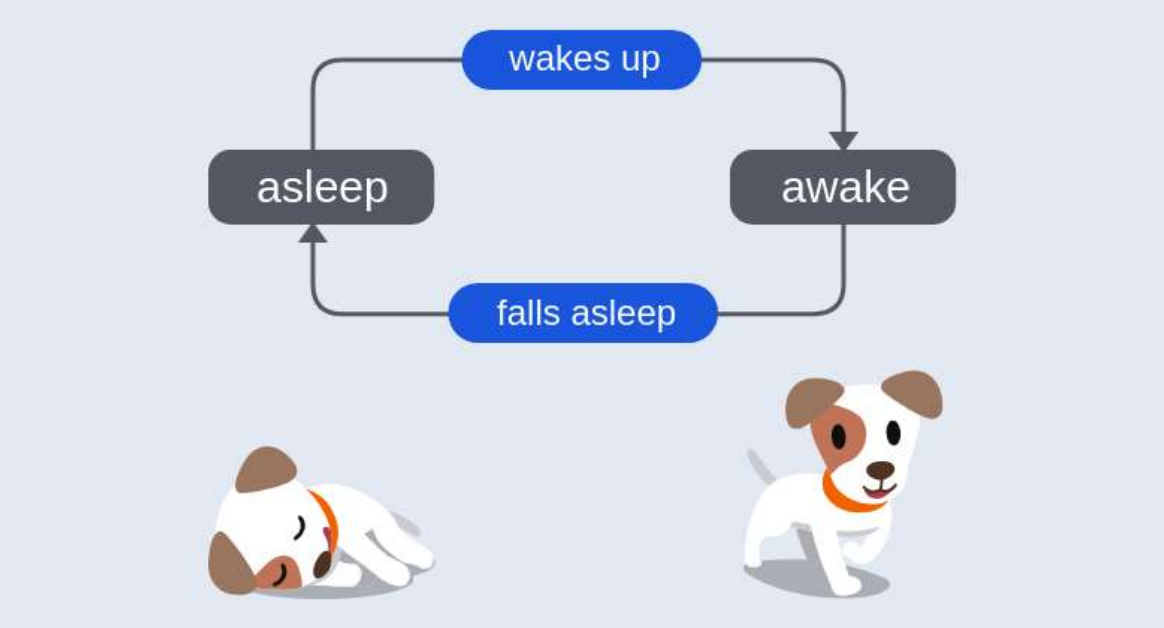


# Назначение диаграммы состояний

Диаграммы состояний служат для моделирования поведения реактивных объектов.

**Реактивный объект** - объект, поведение которого лучше всего характеризуется его реакцией на события, произошедшие вне его собственного контекста. У реактивного объекта есть четко выраженный жизненный цикл, когда текущее поведение обусловлено прошлым.

Диаграммы состояний могут существовать автономно, и могут использоваться для моделирования поведения классов, прецедентов и системы в целом.



# Диаграмма состояний

**Диаграмма состояний (state machine diagram)** — это диаграмма, которая показывает автомат, фокусируя внимание на его потенциальных состояниях и переходах между ними.

**Диаграммы состояний служат для моделирования поведения реактивных объектов.**

Диаграммы состояний могут существовать автономно, и могут использоваться для моделирования поведения классов, прецедентов и системы в целом

# Диаграмма состояний может содержать

- Состояния
- Переходы
- Примечания

# Состояние

**Состояние объекта (state)** — это ситуация в его жизни, на протяжении которой он удовлетворяет некоторому условию, осуществляет определенную деятельность или ожидает какого-то события.

Состояние определяют следующие элементы:

**имя** - текстовая строка, которая отличает одно состояние от всех остальных. Имя состояния должно быть уникальным внутри объемлющего пакета

**внутренние действия** - действия, выполняемые при входе в состояние и выходе из него, а также выполняемые без выхода из состояния

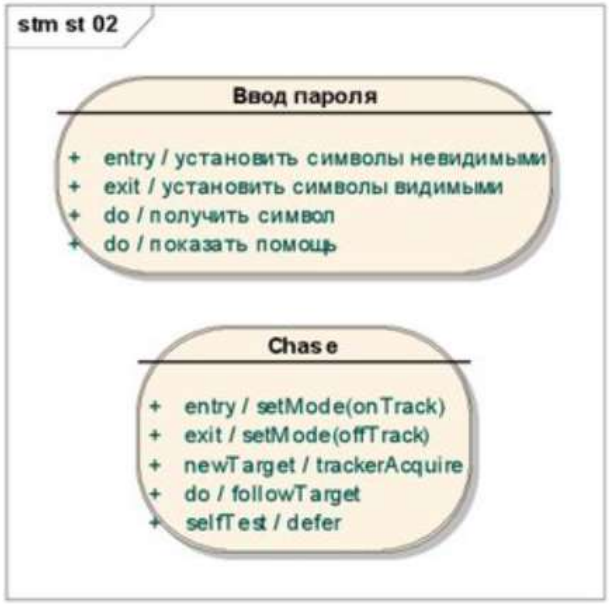
**подсостояния** - внутри состояния могут существовать подсостояния, как непересекающиеся (активизируемые последовательно), так и параллельные (активные одновременно)

**отложенные события** - список событий, которые не обработаны в этом состоянии, а отложены и поставлены в очередь для обработки объектом в некотором другом состоянии.

# Внутреннее действие

**Внутреннее действие** — текстовая строка формата

***<метка действия / выражение действия>***




Перечень меток действия имеет фиксированные значения, которые не могут быть использованы в качестве имен событий

**entry** — указывает на действие, которое выполняется в момент входа в данное состояние

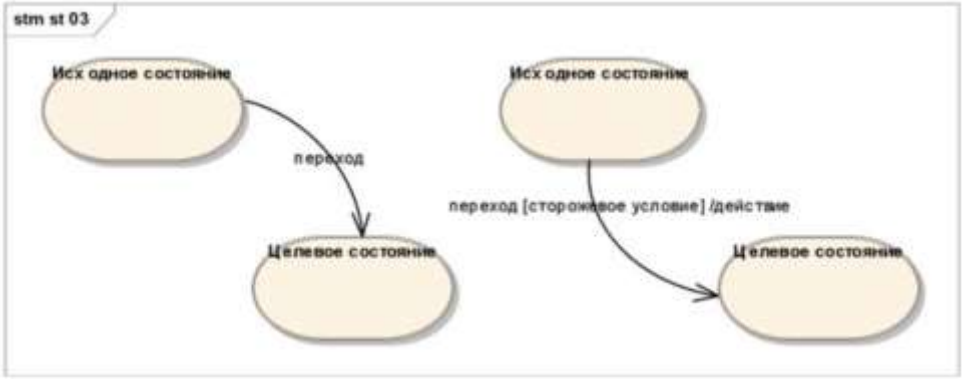
**exit** — указывает на действие, которое выполняется в момент выхода из данного состояния

**do** — специфицирует выполняющуюся деятельность, которая выполняется в течение всего времени, пока объект находится в данном состоянии, или до тех пор, пока не закончится вычисление, специфицированное следующим за ней выражением действия

**include** — используется для обращения к подавтомату, при этом следующее за ней выражение действия содержит имя этого подавтомата

# Переход

**Переход (transition)** — это отношение между двумя состояниями, показывающее, что объект, находящийся в **исходном состоянии**, должен выполнить определенное **действие** и перейти в **целевое состояние**, как только произойдет указанное **событие** и будут выполнены определенные **сторожевые условия.**

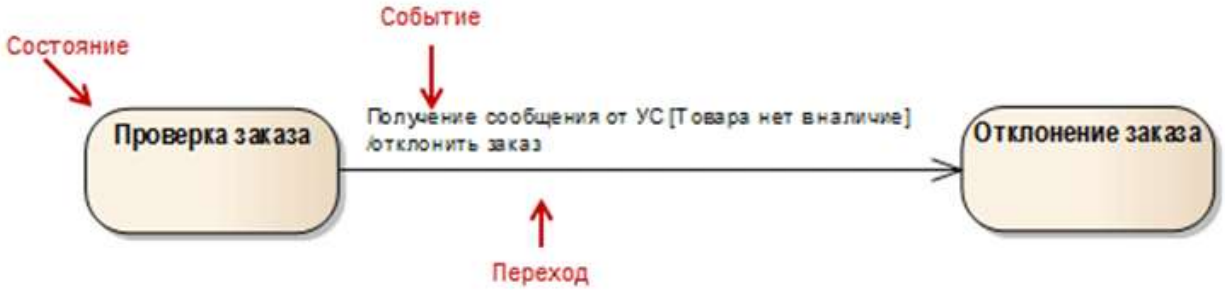



Каждый переход может помечен строкой текста, которая имеет следующий общий формат:

***<сигнатура события>[<сторожевое условие>] <выражение действия>***

Сигнатура события описывает некоторое событие с необходимыми аргументами:

***<имя события>(<список параметров, разделенных запятыми>)***



# Событие-триггер, сторожевое условие и действие

**Событие-триггер (trigger event)** — событие, при получении которого объектом, находящимся в исходном состоянии, может сработать переход

**Сторожевое условие (guard condition)** — логическое выражение, которое вычисляется при возникновении события-триггера. Если значение истинно, то переходу разрешено сработать, если ложно - переход не срабатывает

**Действие (action)** — это атомарное вычисление, которое выполняется в том случае, когда переход срабатывает, и приводит к изменению состояния объекта или возврату значения

# Псевдосостояние

- **Псевдосостояние** (pseudo-state) - вершина в конечном автомате, которая имеет форму состояния, но не обладает поведением.
- **Начальное состояние** (start state) - разновидность псевдосостояния, обозначающее начало выполнения процесса изменения состояний конечного автомата или нахождения моделируемого объекта в составном состоянии.
- **Конечное состояние** (final state) - разновидность псевдосостояния, обозначающее прекращение процесса изменения состояний конечного автомата или нахождения моделируемого объекта в составном состоянии.

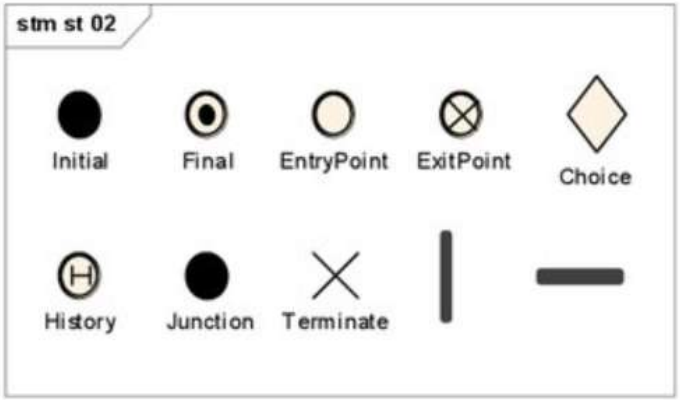
(а) (б)

начальное состояние конечное состояние

# Псевдосостояния

**Псевдосостояние объекта (pseudo-state)** — абстракция для различных типов вершин, используемая в диаграмме состояний

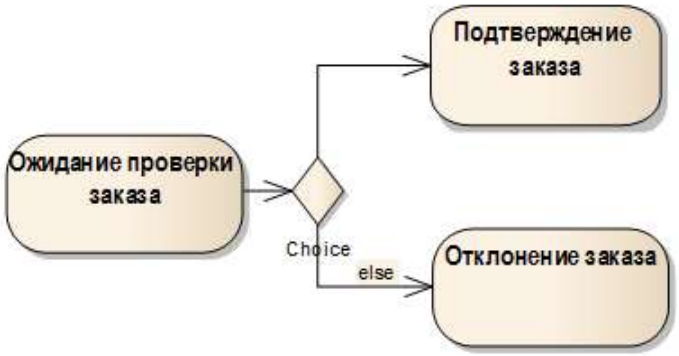
Псевдосостояния не содержат никаких внутренних действий




- **Начальное состояние (initial)**
- **Конечное состояние (final)**
- **Точка входа (entry point)**
- **Точка выхода (exit point)**
- **Выбор (choice)**
- **Историческое состояние (history)**
- **Соединение (junction)**
- **Прекращение (terminate)**
- **Ветвление (fork)**

# Примеры

Создание заказа
Проверка заказа
Отправка заказа
Закрытие заказа
Снятие заказа
Удаление заказа
Terminate


Подтверждение заказа
Ожидание проверки заказа
Choice
else
Отклонение заказа

# Составные состояния и подсостояния

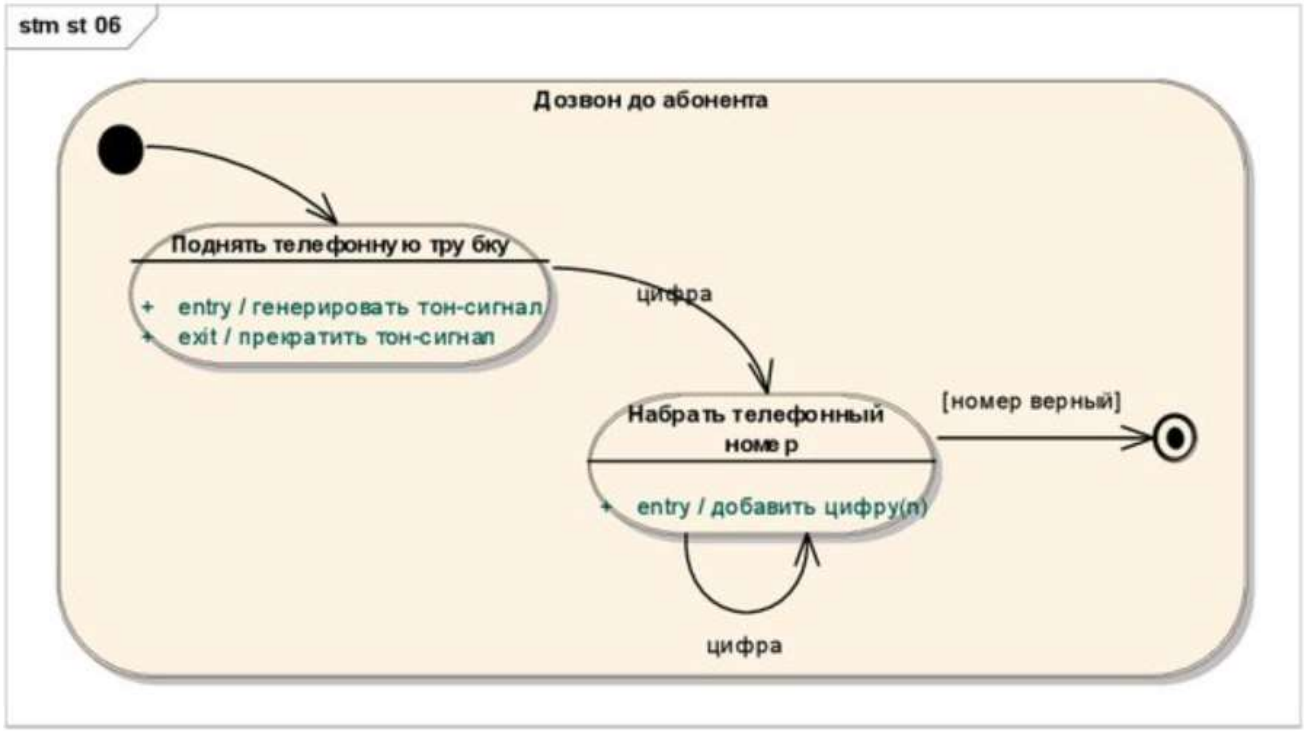
**Простое состояние** — состояние, не имеющее внутренней структуры

**Составное состояние (composite state)** — сложное состояние, которое состоит из других вложенных в него состояний

**Подсостояние (substate)** — состояние, являющееся частью другого состояния

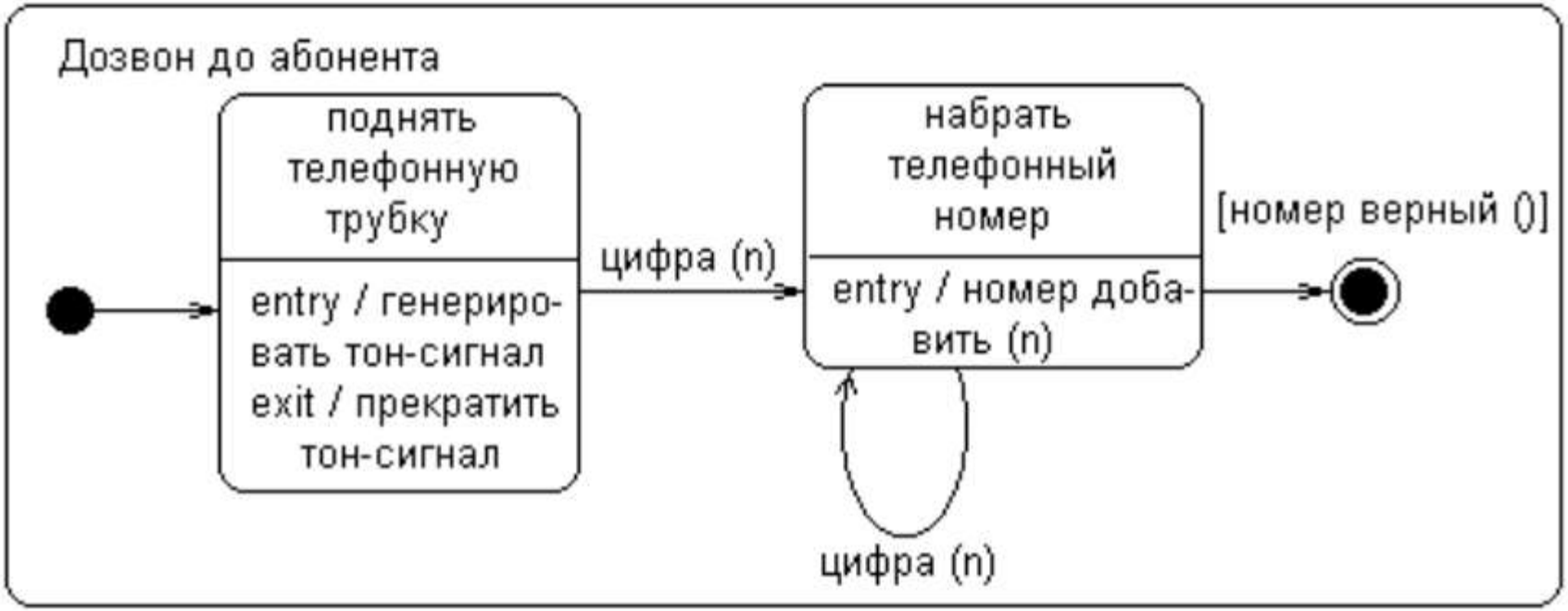
# Последовательные подсостояния

**Последовательные подсостояния (sequental substates)** используются для моделирования такого поведения объекта, во время которого в каждый момент времени объект может находиться в одном и только одном подсостояний



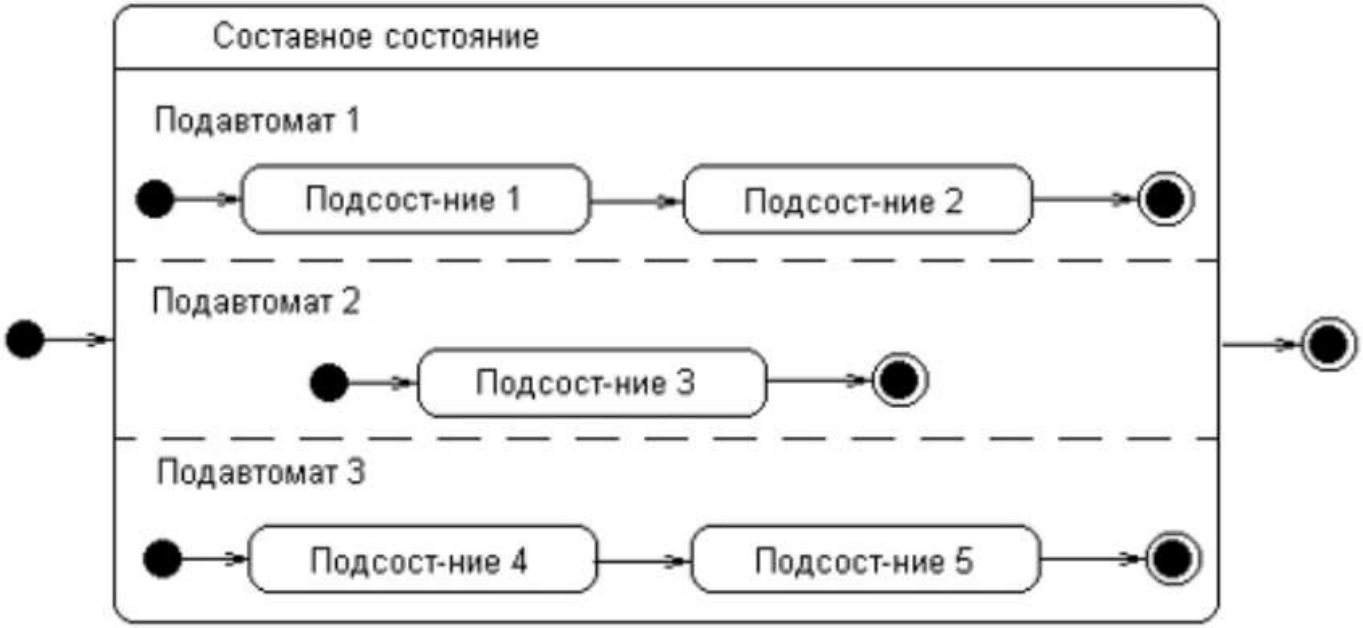

Начальное и конечное состояния объекта должны быть единственными в каждом составном состоянии

# Пример


Дозвон до абонента
поднять телефонную трубку
entry / генерировать тон-сигнал
exit / прекратить тон-сигнал
цифра (n)
набрать телефонный номер
entry / номер добавить (n)
цифра (n)
[номер верный ()]

# Параллельные подсостояния

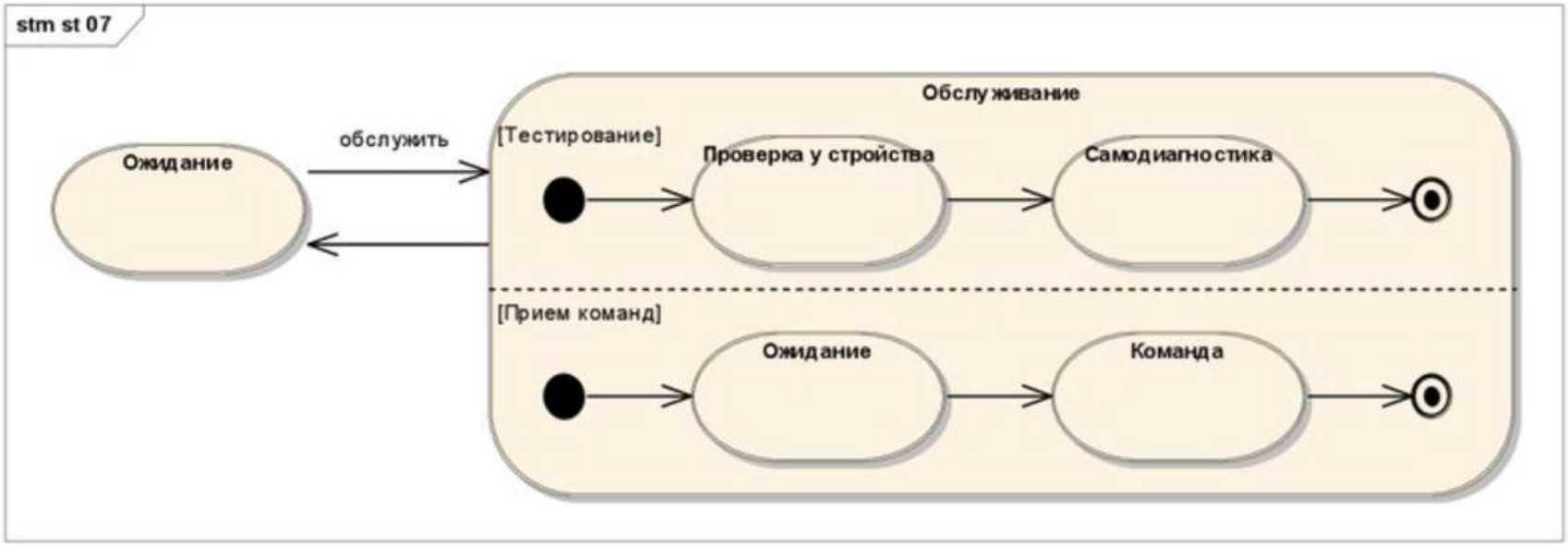
**Параллельные подсостояния (concurrent substates)** позволяют специфицировать два или более автомата, которые выполняются параллельно в контексте объемлющего объекта




- Объемлющий объект может одновременно находиться в каждом из параллельных подсостояний
- Если какой-либо из подавтоматов пришел в свое конечное состояние раньше других, то он должен ожидать, пока другие подавтоматы не придут в свои конечные состояния

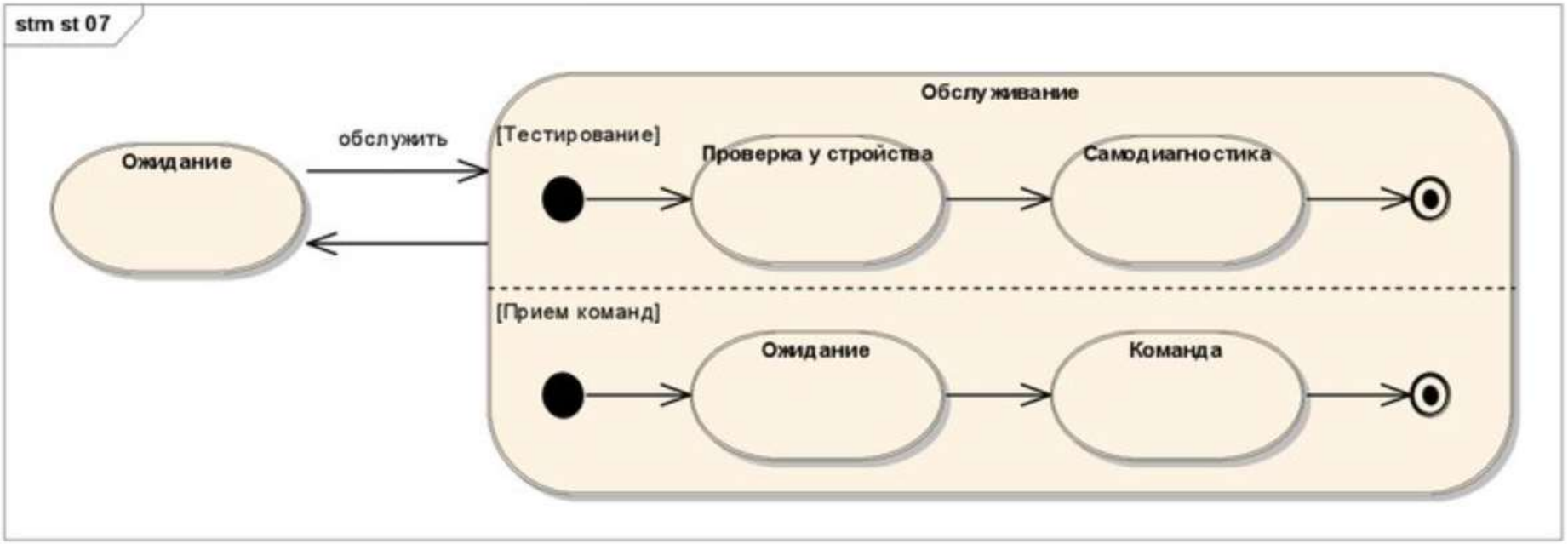
# Параллельные подсостояния

**Параллельные подсостояния (concurrent substates)** позволяют специфицировать два или более автомата, которые выполняются параллельно в контексте объемлющего объекта




- Объемлющий объект может одновременно находиться в каждом из параллельных подсостояний
- Если какой-либо из подавтоматов пришел в свое конечное состояние раньше других, то он должен ожидать, пока другие подавтоматы не придут в свои конечные состояния

# Параллельные подсостояния

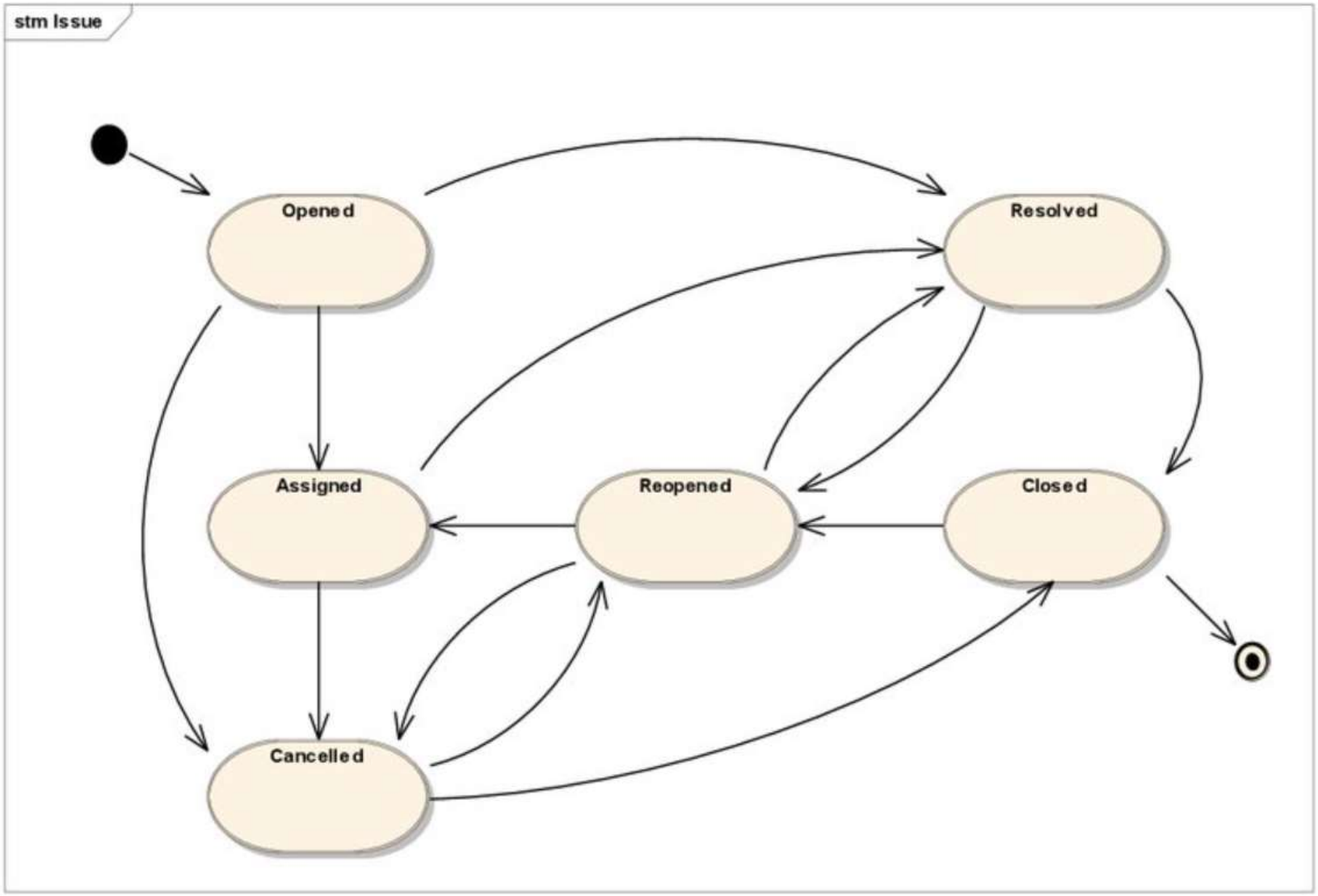


# Советы

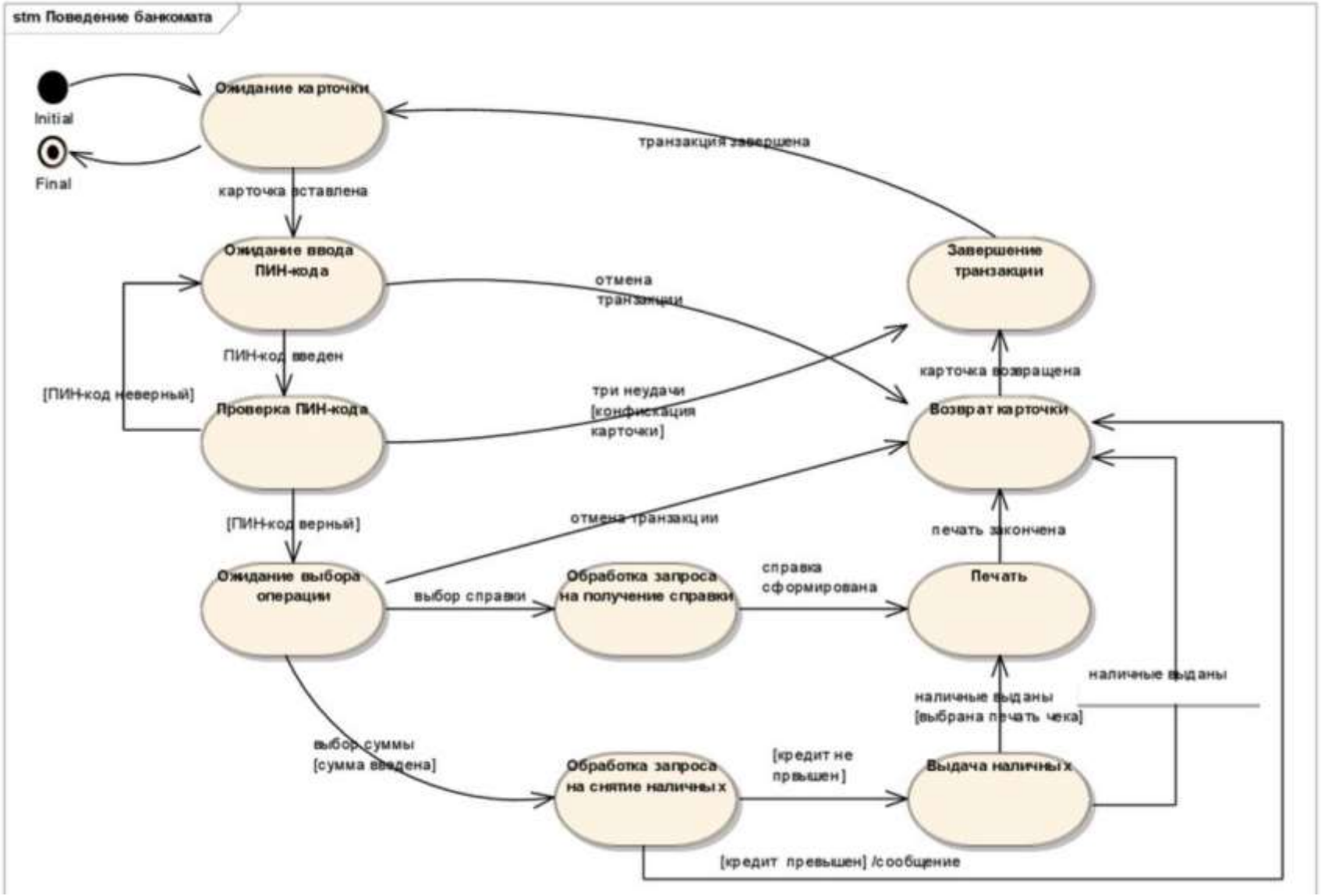
При моделировании состояний соблюдайте следующие правила:

- Дайте диаграмме имя, соответствующее назначению
- Сначала смоделируйте устойчивые состояния объекта, затем переходите к допустимым переходам состояний
- Ветвления и параллельность моделируйте на отдельной диаграмме
- Располагайте элементы так, чтобы число пересечений было минимальным

# Примеры


stm Issue
Opened
Resolved
Assigned
Reopened
Closed
Cancelled

# Примеры


stm Поведение банкомата
Initial
Final
Ожидание карточки
транзакция завершена
карточка вставлена
Ожидание ввода ПИН-кода
отмена транзакции
Завершение транзакции
ПИН-код введен
карточка возвращена
[ПИН-код неверный]
Проверка ПИН-кода
три неудачи [конфискация карточки]
Возврат карточки
[ПИН-код верный]
отмена транзакции
печать закончена
Ожидание выбора операции
выбор справки
Обработка запроса на получение справки
справка сформирована
Печать
наличные выданы
наличные выданы [выбрана печать чека]
выбор суммы [сумма введена]
Обработка запроса на снятие наличных
[кредит не превышен]
Выдача наличных
[кредит превышен] /сообщение